EXPERIMENT 6

Academic Year: 2026-27 Year and Sem: TE V
Subject: DEV Lab Course Code: 2175119

PROBLEM STATEMENT:

To Implement Containerization using Docker

Hardware / Software Required: Ubuntu operating system, An active AWS account. AWS Management

THEORY:

Introduction to Docker

Docker is an open-source platform that enables developers to build, ship, and run applications in lightweight, portable containers. It simplifies application deployment by packaging software along with all its dependencies into a standardized unit, ensuring consistency across different environments.

Why Use Docker?

1. Portability: Containers run consistently across different environments (development testing, and production).

2. Lightweight: Containers share the host OS kernel, reducing overhead compared to virtual machines.

3. Scalability: Applications can be easily scaled up or down using container orchestration tools.

4. Rapid Deployment: Containers can be quickly created, started, stopped, and destroyed.

5. Dependency Management: All dependencies are included within the container, eliminating compatibility issues.

Key Docker Components

1. Docker Engine

● The core component that runs and manages containers.

● Consists of the Docker daemon, REST API, and CLI (Command-Line Interface).

2. Docker Images

● A read-only template containing the application code, dependencies, and OS libraries.

● Created using Dockerfiles and stored in Docker Hub or private registries.

3. Docker Containers :

A runnable instance of a Docker image.

● Provides process isolation and runs in a lightweight, virtualized environment.

4. Docker Hub

A cloud-based registry for storing and sharing Docker images.

● Public and private repositories are available.

5. Docker Compose

● A tool for defining and running multi-container applications using a YAML file.

● Simplifies managing complex applications with multiple services.

6. Docker Volumes

● Used for persistent storage, allowing data to be shared between containers and retained after container termination.

How Docker Works

1. Developers write code and define dependencies in a Dockerfile.

2. Docker builds an image from the Dockerfile.

3. A container is launched from the image using the docker run command**.**

4. Docker manages networking to connect containers together.

5. Containers can be scaled, updated, and deployed across multiple platforms.

Step 1: Update Package List

Command

sudo apt update

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ sudo apt update
[sudo] password for apsit:
Get:1 https://dl.google.com/linux/chrome/deb stable InRelease [1,825 B]
Get:2 https://dl.google.com/linux/chrome/deb stable/main amd64 Packages [1,210 B]
Hit:3 http://in.archive.ubuntu.com/ubuntu jammy InRelease
Get:4 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:5 http://in.archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:6 http://in.archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:7 http://in.archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [3,285 kB]
Get:8 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [3,023 kB]
Get:9 http://in.archive.ubuntu.com/ubuntu jammy-updates/main i386 Packages [961 kB]
Get:10 http://in.archive.ubuntu.com/ubuntu jammy-updates/main amd64 DEP-11 Metadata [114 kB]
Get:11 http://security.ubuntu.com/ubuntu jammy-security/main i386 Packages [778 kB]
Get:12 http://in.archive.ubuntu.com/ubuntu jammy-updates/main amd64 c-n-f Metadata [19.2 kB]
Get:13 http://security.ubuntu.com/ubuntu jammy-security/main amd64 DEP-11 Metadata [54.5 kB]
Get:14 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 DEP-11 Metadata [208 B]
Get:15 http://in.archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 DEP-11 Metadata [212 B]
```

Step 2: Install Prerequisite Packages

sudo apt install ca-certificates curl gnupg

Install a few prerequisite packages which let apt use packages over HTTPS:

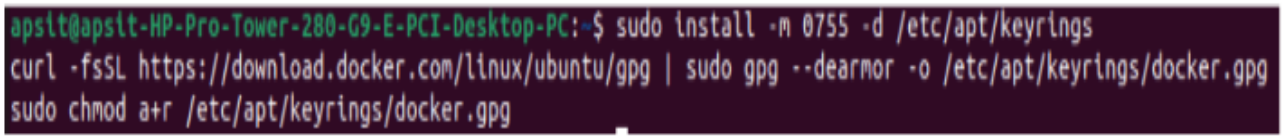
```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ sudo apt install ca-certificates curl gnupg
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ca-certificates is already the newest version (20240203~22.04.1).
ca-certificates set to manually installed.
gnupg is already the newest version (2.2.27-3ubuntu2.5).
gnupg set to manually installed.
The following NEW packages will be installed:
  curl
0 upgraded, 1 newly installed, 0 to remove and 154 not upgraded.
Need to get 194 kB of archives.
After this operation, 455 kB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://in.archive.ubuntu.com/ubuntu jammy-updates/main amd64 curl amd64 7.81.0-1ubuntu1.23 [194 kB]
Fetched 194 kB in 5s (40.0 kB/s)
Selecting previously unselected package curl.
(Reading database ... 218779 files and directories currently installed.)
Preparing to unpack .../curl_7.81.0-1ubuntu1.23_amd64.deb ...
Unpacking curl (7.81.0-1ubuntu1.23) ...
Setting up curl (7.81.0-1ubuntu1.23) ...
Processing triggers for man-db (2.10.2-1) ...
```

Step 3: Add Docker GPG Key

sudo install -m 0755 -d /etc/apt/keyrings

curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

sudo chmod a+r /etc/apt/keyrings/docker.gpg



Purpose :Adds Docker's official security key to verify downloaded packages.

Step 4: Add Docker Repository

echo \

"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu \

$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \

sudo tee /etc/apt/sources.list.d/docker.list > /dev/null



Update Package Database :



Step 5: Verify Repository

apt-cache policy docker-ce

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ apt-cache policy docker-ce
docker-ce:
  Installed: (none)
  Candidate: 5:29.3.0-1~ubuntu.22.04~jammy
  Version table:
     5:29.3.0-1~ubuntu.22.04~jammy 500
        500 https://download.docker.com/linux/ubuntu jammy/stable amd64 Packages
     5:29.2.1-1~ubuntu.22.04~jammy 500
        500 https://download.docker.com/linux/ubuntu jammy/stable amd64 Packages
     5:29.2.0-1~ubuntu.22.04~jammy 500
        500 https://download.docker.com/linux/ubuntu jammy/stable amd64 Packages
     5:29.1.5-1~ubuntu.22.04~jammy 500
        500 https://download.docker.com/linux/ubuntu jammy/stable amd64 Packages
     5:29.1.4-1~ubuntu.22.04~jammy 500
```

Step 6: Install Docker

sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  docker-ce-rootless-extras git git-man liberror-perl pigz slirp4netns
Suggested packages:
  cgroupfs-mount | cgroup-lite docker-model-plugin git-daemon-run | git-daemon-sysvinit git-doc git-email git-gui gitk gitweb git-cvs git-mediawiki git-svn
The following NEW packages will be installed:
  containerd.io docker-buildx-plugin docker-ce docker-ce-cli docker-ce-rootless-extras docker-compose-plugin git git-man liberror-perl pigz slirp4netns
0 upgraded, 11 newly installed, 0 to remove and 154 not upgraded.
Need to get 101 MB of archives.
After this operation, 414 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y
```

Step 7: Check Docker Status

sudo systemctl status docker

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
     Loaded: loaded (/lib/systemd/system/docker.service; enabled; vendor preset: enabled)
     Active: active (running) since Fri 2026-03-13 15:51:20 IST; 27s ago
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
   Main PID: 9991 (dockerd)
      Tasks: 18
     Memory: 25.7M
        CPU: 455ms
     CGroup: /system.slice/docker.service
             └─9991 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Mar 13 15:51:19 apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC dockerd[9991]: time="2026-03-13T15:51:19.925387587+05:30" level=info msg="Restoring containers: start."
Mar 13 15:51:19 apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC dockerd[9991]: time="2026-03-13T15:51:19.988370788+05:30" level=info msg="Deleting nftables IPv4 rules" erro>
Mar 13 15:51:20 apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC dockerd[9991]: time="2026-03-13T15:51:20.000435942+05:30" level=info msg="Deleting nftables IPv6 rules" erro>
Mar 13 15:51:20 apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC dockerd[9991]: time="2026-03-13T15:51:20.503899760+05:30" level=info msg="Loading containers: done."
Mar 13 15:51:20 apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC dockerd[9991]: time="2026-03-13T15:51:20.517335080+05:30" level=info msg="Docker daemon" commit=83bca51 cont>
Mar 13 15:51:20 apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC dockerd[9991]: time="2026-03-13T15:51:20.517524618+05:30" level=info msg="Initializing buildkit"
Mar 13 15:51:20 apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC dockerd[9991]: time="2026-03-13T15:51:20.546411232+05:30" level=info msg="Completed buildkit initialization"
Mar 13 15:51:20 apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC dockerd[9991]: time="2026-03-13T15:51:20.554874552+05:30" level=info msg="Daemon has completed initializatio>
Mar 13 15:51:20 apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC dockerd[9991]: time="2026-03-13T15:51:20.554936087+05:30" level=info msg="API listen on /run/docker.sock"
Mar 13 15:51:20 apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC systemd[1]: Started Docker Application Container Engine.
lines 1-22/22 (END)
```

Step 8: Run Hello World Container

sudo docker run hello-world

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
ea52d2000f90: Download complete
Digest: sha256:85404b3c53951c3ff5d40de0972b1bb21fafa2e8daa235355baf44f33db9dbdd
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Check Docker version

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ docker --version
Docker version 29.3.0, build 5927d80
```

Pull an Image from Docker Hub:

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
01d7766a2e4a: Pull complete
fd8cda969ed2: Download complete
Digest: sha256:d1e2e92c075e5ca139d51a140fff46f84315c0fdce203eab2807c7e495eff4f9
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest
```

Step 9: List Images

docker images

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ docker images
                                                    i Info →  U  In Use
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest   85404b3c5395       25.9kB         9.52kB    U
ubuntu:latest        d1e2e92c075e        119MB          31.7MB
```

Run a Container :

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ docker run -it ubuntu
root@045dea8ba81f:/#
```

Step 10 : Install Software inside the container

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ docker run -it ubuntu
root@045dea8ba81f:/# apt update
apt install nodejs
```

Verify Installation :

```
root@045dea8ba81f:/# node -v
v18.19.1
```

List Running Container :

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

List All Containers :

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ docker ps -a
CONTAINER ID   IMAGE         COMMAND       CREATED         STATUS                     PORTS     NAMES
045dea8ba81f   ubuntu        "/bin/bash"   3 minutes ago   Exited (0) 42 seconds ago            busy_germain
95f1baf0d39e   hello-world   "/hello"      6 minutes ago   Exited (0) 6 minutes ago             sad_williams
```

View the Latest Container :

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ docker ps -l
CONTAINER ID   IMAGE     COMMAND       CREATED          STATUS                          PORTS     NAMES
045dea8ba81f   ubuntu    "/bin/bash"   4 minutes ago    Exited (0) About a minute ago             busy_germain
```

Start a Stopped Container :

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ docker start 045dea8ba81f
045dea8ba81f
```

Stop a Container :

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ docker stop 045dea8ba81f
045dea8ba81f
```

Remove a Container:

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ docker rm 045dea8ba81f
045dea8ba81f
```

Task 2 : Creating and Running a Dockerized Python Flask Application

Create Application Folder :

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ mkdir myapp
cd myapp
```

Create The flask application :

nano app.py

app.py
~/myapp

```python
from flask import Flask
app = Flask(__name__)

@app.route("/")
def hello():
    return "Hello from Docker Container!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Create requirements.txt

nano requirements.txt

requirements.txt
~/myapp

app.py

```
flask
requests
numpy
```

Create Dockerfile :

nano dockerfile

dockerfile
~/myapp

app.py | requirements.txt

```
FROM python:3.9

WORKDIR /app

COPY . /app

RUN pip install -r requirements.txt

CMD ["python", "app.py"]

```

Build Docker Image :

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/myapp$ docker build -t myapp .
[+] Building 31.7s (4/8)                                                
```

CONCLUSION:

Understanding Docker's architecture and the container life cycle is fundamental for harnessing the full potential of containerization in software development. By installing Docker and executing key commands, this experiment empowers participants to manage Docker images, create containers, and interact with them seamlessly. The knowledge gained in this experiment lays a solid foundation for incorporating Docker into the software development workflow, promoting consistency and scalability across diverse computing environments.